

Laboratory Notes

Week 4 Lab: Publishers and Subscribers

1 Aim

This laboratory aims to develop the student's understanding of Publishers, Subscribers and Messages in ROS. It involves developing specific publisher and subscriber nodes and examining how they interact. The laboratory will cover the following specific areas:

- Developing a new Publisher node.
- Developing a new Subscriber node.
- Making a new communication Topic.
- Combining Publishers and Subscribers in the same Node.

2 Background

The fundamental method for communication in ROS is through the concept of Topics. Nodes that need to share information will Publish messages to an appropriate topic, while Nodes that need to receive that information will Subscribe to the same topic. This lab explores the concept of Topics through building both a Publisher Node and a Subscriber Node in python. We will also make a new Topic which we will use for communication between our publisher and subscriber nodes. Throughout the academic session we will be building larger nodes to accomplish specific autonomous tasks – understanding of Topics, along with how to publish and subscribe to these topics will be vital in successfully completing the larger tasks throughout the session.

3 Procedure

Using the package that we built in the Week 3 laboratory, we will develop two new nodes. The first node is a simple publisher, which will periodically publish the location of a robot. Similarly, the second node will be a simple subscriber, which will read the robot location from the same topic and print it to the screen. We will also make a new topic called *robot_location*, which will receive messages of type *geometry_msgs/Point*. If you look up the details of this message type, you should observe that it has the following structure:

- float64 x
- float64 y
- float64 z

We can see that this message contains three double-precision floating-point numbers corresponding to the locations on the X, Y and Z axes. Using the package that you created in the Week 3 laboratory, setup two new files inside the code directory (the one with the same name as the package) - *publisher_node.py* and *subscriber_node.py*. If you do not already have this package, you can follow the instructions from the Week 3 laboratory to create it now.

3.1 Building the Publisher Node

First, we will build a publisher node. Open your new *publisher_node.py* file in VS Code and populate the file with the following code:

```
1 import rclpy
2 from rclpy.node import Node
3 from geometry_msgs.msg import Point
4
5 class MyPublisher(Node):
6     def __init__(self):
7         # setup node and publisher
8         super().__init__('robot_location_publisher')
9         self.pub = self.create_publisher(Point, 'robot_location', 10)
10        # initialise the location of the robot
11        self.location = Point()
12        self.location.x = 10.0
13        self.location.y = 10.0
14        self.location.z = 10.0
15        # setup a timer to update the position at a rate of 10 Hz
16        timer_period = 0.1
17        self.timer = self.create_timer(timer_period, self.timer_callback)
18
19    def timer_callback(self):
20        # publish location
21        self.pub.publish(self.location)
22        self.get_logger().info('Publishing location: %f, %f' % (self.location.x, self.location.y))
23        # Simulate a move
24        self.location.x += 1.0
25        self.location.y += 2.0
26
27    def main(args=None):
28        rclpy.init(args=args)
29        robot_publisher = MyPublisher()
```

```

30     rclpy.spin(robot_publisher)
31
32 if __name__ == '__main__':
33     main()

```

This node has four main parts. Firstly, we have the import statements at the top. You can see that this script imports the **rclpy package**, which is the main interface for interacting with ROS from a Python script. We also import the **Point class** from the *geometry_msgs.msg* package. This is the definition of the *geometry_msgs/Point* message type we want to publish. Following the import statements, the next two sections are contained within the definition of our class, which is called *MyPublisher*. When you are setting up a node in ROS2, you need to make your class a subclass of the existing ROS2 class called *Node*. The class then begins with a constructor method, which is a special method that is automatically called when an object of the class is instantiated. Constructors in python are always called `__init__(self)`. All initialisation is generally performed inside the constructor. The first line in our constructor calls the constructor of the parent (*Node*) class, we pass the name of our new node as an argument within this call:

```

8     super().__init__('robot_location_publisher')

```

This call to the constructor of the *Node* class, sets up the new node in ROS – thus allowing communication with other nodes. The second line in our constructor:

```

9     self.pub = self.create_publisher(Point, 'robot_location', 10)

```

declares that this node will be publishing to the topic called *robot_location*, which uses messages of type *Point* (i.e. *geometry_msgs/Point*) and has a queue size of 10. The queue size tells us the maximum number of messages that should be kept. This is a queue, so it is first-in first-out in structure. Our publisher is set up as a special publisher object (via calling the node method *create_publisher*). In this script we have called our new object *self.pub*. The next lines of our constructor initialise the message that we want to publish to the *robot_location* topic.

```

11     self.location = Point()
12     self.location.x = 10
13     self.location.y = 10
14     self.location.z = 10

```

This code first instantiates an object of the *Point* class (called *self.location*), then stores initial values of 10 in each of the x, y, and z member variables. The final part of our constructor sets up a timer:

```

16     timer_period = 0.1
17     self.timer = self.create_timer(timer_period, self.timer_callback)

```

Here we have first set a variable to represent the period of the timer (in this case 100ms) and then called the *create_timer* method of the *Node* class. The *create_timer* method requires 2 arguments: the first is the period of the timer and the second is the name of a callback method. Callbacks are used a lot in ROS2 and are special methods that are called whenever certain internal events happen – in this case whenever the timer reaches its given period. We have named our callback *self.timer_callback*.

The next section of code is the callback method itself. As this is automatically called every time the timer reaches its given period, this is where we are going to actually publish our message:

```
21 self.pub.publish(self.location)
22 self.get_logger().info('Publishing location: %f, %f' %(self.location.x, self.location.y))
```

Publishing is done via a call to the *publish* method of our publisher object, which requires the message (to be published) as an argument. The second line is useful for debugging and simply prints a message to this nodes INFO stream. The last two lines of code in the callback method update the values stored in our location object.

The last section of code is our “main” function. This contains three lines:

```
28 rclpy.init(args=args)
29 robot_publisher = MyPublisher()
30 rclpy.spin(robot_publisher)
```

The first line here initialises all ROS2 communication. We then instantiate an object of our new *MyPublisher()* class. The final line *rclpy.spin(robot_publisher)* blocks any further execution of the main function until the robot_publisher shuts down, thus allowing the node to continue publishing until we kill the node.

There are also a couple of lines at the very bottom:

```
32 if __name__ == '__main__':
33     main()
```

this is a standard python way to ensure that the main function only executes when this script is the entry point of a program. Hence, if another script is importing our code as a module, *main()* will not execute.

3.2 Building the Subscriber Node

Next, we will build a subscriber node. Add the following code to *subscriber_node.py*:

```
1 import rclpy
2 from rclpy.node import Node
3 from geometry_msgs.msg import Point
4
5 class MySubscriber(Node):
6     def __init__(self):
7         super().__init__('robot_location_subscriber')
8         self.subscription = self.create_subscription(Point, 'robot_location', self.location_callback, 10)
9         self.subscription
10
11     def location_callback(self, location):
12         self.get_logger().info('Received robot location: %f, %f' %(location.x, location.y))
13
14 def main(args=None):
15     rclpy.init(args=args)
16     robot_location_subscriber = MySubscriber()
17     rclpy.spin(robot_location_subscriber)
```

```

18 robot_location_subscriber.destroy_node()
19 rclpy.shutdown
20
21 if __name__ == '__main__':
22     main

```

This code again has four sections. The first (the imports) and last (main function) are essentially the same as in the publisher node. The main differences here are within the class we are defining. The constructor of the class again calls the constructor of the parent (Node) class and passes the name of our new node as an argument. The next line in our constructor sets up our subscriber:

```

8 self.subscription = self.create_subscription(Point, 'robot_location', self.location_callback, 10)

```

This instantiates a special subscriber object (via calling the node method *create_subscription*). In this script we have called our new object *self.subscription*. The arguments to *create_subscription* are the message type (Point), the topic name ('robot_location'), a callback method to run whenever a message is received (self.location_callback) and the queue size (10). The line of code after instantiating the subscription object (which just repeats the name of the object) is not necessary – it is just present to suppress any warnings about an unused object, as self.subscription is not referenced anywhere else in the script.

The last part of our subscriber node is the callback method. In our script, the callback is just used to print the received location message to the INFO stream, much like the publisher node does before sending the message. This will allow us to verify that the messages are being sent and received.

3.3 Running the Nodes

Once you have finished coding both nodes, you should edit the *setup.py* file in your package to add entry points for running the nodes (remember to **substitute** the name of your package for *jeffs_pkg*):

```

1 'publisher_node = jeffs_pkg.publisher_node:main',
2 'subscriber_node = jeffs_pkg.subscriber_node:main'

```

Now navigate to the root workspace directory (ros2_ws) in a terminal and re-build the package using *colcon build*. You should now be able to run your two nodes in separate terminals:

```

1 ros2 run jeffs_pkg publisher_node
2 ros2 run jeffs_pkg subscriber_node

```

3.4 Subscribers, Publishers and Messages

Using the Gazebo world *blocked_road*, write a new python script (in your package) that makes the TB3 move towards the wall, then stops when approximately 1m from the wall.

Steps:

1. Write a publisher that writes to the /cmd_vel topic in order to move the robot.
2. Write a subscriber that reads from the /scan topic to determine the distance to the wall

3. Depending on the data received, change the data sent to the `/cmd_vel` topic.

Tips:

- The code provided in the Week 3 laboratory will provide the basis for this code.
- You should only use 1 script file – you can have publishers and subscribers in the same node.
- If you get stuck, please speak to your lab demonstrator, they can provide advice throughout.

4 Summary

In today's laboratory we have explored the concepts of Topics in ROS, via setting up a new Topic and developing Publisher and Subscriber nodes to communicate over this topic. We also looked at how to combine publishers and subscribers within the same node in order to achieve specific tasks. We will continue to build upon our knowledge of publishers and subscribers throughout the session, to achieve more complex ROS tasks. Next week we will take our first look at SLAM and the broader area of Navigation in ROS.